Online Voting System

Project Report

A Python-based CLI application for secure, multi-user electronic voting with real-time result tracking.

Mini Project | Python | File I/O | CLI

Project Overview

Purpose

Provide a simple, secure digital platform for conducting elections among registered users with anti-duplicate voting.

Language

Python 3.x — pure standard library. No third-party packages required. Runs on any OS with Python installed.

Storage

Flat-file persistence using users.txt for voter registry and votes.txt for recording cast votes.

Interface

Command-line interface (CLI) with a numbered menu system guiding users through voting and result viewing.

System Architecture

main.py

Application Entry Point

Contains all functions and the main menu loop. Handles user input/output and coordinates all modules.

users.txt

Voter Registry

Stores one user ID per line. Pre-loaded with 10 registered voters (user1 – user10).

votes.txt

Vote Ledger

Appended at runtime. Each line stores a user–choice pair in CSV format: userID,candidateKey

Key Features

01

User Authentication

Every voter is validated against users.txt before being allowed to vote. Invalid IDs are rejected immediately.

02

Duplicate Vote Prevention

votes.txt is parsed before each vote. If a user ID already appears, the system blocks the duplicate attempt.

03

Three-Candidate Ballot

Supports Alice, Bob, and Charlie. Menu keys (1/2/3) are validated — invalid inputs are rejected gracefully.

04

Persistent Storage

Votes are appended to disk immediately, surviving process restarts. No in-memory-only state.

05

Live Results

"Show Results" reads and tallies votes.txt in real time, displaying accurate counts for all candidates.

06

Menu-Driven CLI

Three-option menu loop: Vote, Show Results, Exit. Clean prompts with emoji feedback for user actions.

Module Breakdown

Function	Purpose	Key Logic
`load_users()`	Read voter registry	Opens users.txt, strips whitespace, returns list
`load_votes()`	Read existing votes	Parses votes.txt CSV into {user: choice} dict
`save_vote()`	Persist a new vote	Appends 'user,choice' line to votes.txt
`count_votes()`	Tally vote totals	Iterates vote dict, increments per-candidate counter
`voting_system()`	Core voting flow	Auth → duplicate check → ballot display → record vote
`show_results()`	Display results	Loads + counts votes, prints each candidate's score
`Main loop`	Menu controller	while True loop dispatching options 1 (vote), 2 (results), 3 (exit)

How It Works — Voting Flow

1
Launch
Run main.py
Main menu appears

2
Enter User ID
Validated against users.txt

3
Check Duplicate
ID searched in votes.txt

4
Show Ballot
Candidates 1–3 displayed

5
Record Vote
Saved to votes.txt

✓ Vote Confirmed!

Option 2: Show Results

Reads votes.txt → counts per candidate → displays totals

Sample Data & Outputs

users.txt (10 registered voters)

```
user1
user2
user3
...
user10
```

votes.txt (sample recorded votes)

```
user1,1
user2,2
user3,1
user4,3
user5,2
```

Sample Results Output

```
Alice: 2 votes    Bob: 2 votes    Charlie: 1 vote
```

Limitations & Future Scope

Current Limitations

- Plain-text storage — votes.txt is human-readable and tamperable
- No password/PIN authentication — any user can impersonate another
- Single-session only — no web or GUI interface
- Candidates are hardcoded — changing requires editing source code
- No admin panel to manage elections or reset votes

Future Enhancements

- SQLite or encrypted database for secure vote storage
- Password authentication for voter identity verification
- Web-based GUI using Flask or Django
- Dynamic candidate management via admin interface
- Vote audit log with timestamps and session tracking

Conclusion

The Online Voting System demonstrates practical use of Python fundamentals — file I/O, data structures, and control flow — to solve a real-world problem. While the current version is a solid foundation for a CLI voting tool, it is ready to be extended into a full-featured, secure election platform.

3	10	6	3
Candidates	Registered Users	Core Functions	Project Files